AI VOICE ASSISTANT

CAPSTONE PROJECT

Submitted in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF SCIENCE

IN

Computer science with Cognitive System

Submitted by

HARI NITHIN S – 24BCG017

PRAVEEN KUMAR M- 24BCG040

Under the Guidance of

Dr.V. VALARMATHI **MCA ., M.Phil ., SET ., NET ., Ph.D.**

Department of IT and Cognitive Systems

SRI KRISHNA ARTS AND SCIENCE COLLEGE

COIMBATORE – 641008

OCTOBER 2025

SRI KRISHNA ARTS AND SCIENCE COLLEGE

(Affiliated to Bharathiar University, Kuniamuthur, Coimbatore - 641008)

CERTIFICATE

This is to certify that the Capstone Project report entitled **"AI VOICE ASSISTANT"** in partial fulfilment of requirements for the degree of Bachelor of Science in Computer Science with Cognitive System is a record of bonafide work carried out by **HARI NITHIN S – 24BCG017 , PRAVEEN KUMAR M – 24BCG040** and that no part of this has been submitted for the award of any other degree or diploma and the work has not been published in popular journal or magazine.

GUIDE **HOD & DEAN**

This Capstone Report is submitted for the viva voce conducted on _____ at Sri Krishna Arts and Science College.

INTERNAL EXAMINER **EXTERNAL EXAMINER**

SRI KRISHNA ARTS AND SCIENCE COLLEGE

(Affiliated to Bharathiar University, Kuniamuthur, Coimbatore - 641008)

DECLARATION

I hereby declare that the Capstone Project report entitled **"AI VOICE ASSISTANT**" submitted in partial fulfilment of the requirements for the award of degree of **Bachelor of Science in Computer Science with Cognitive System** is an original work and it has not been previously formed the basis for the award of any Degree, Diploma, Associate ship, Fellowship or similar titles to any other university or body during the period of my study.

Place: Coimbatore

Date:

Signature of the Candidate

HARI NITHIN S

PRAVEEN KUMAR M

ACKNOWLEDGEMENT

I am ineffably indebted to **Dr. K. Sundararaman, M.Com., M.Phil., Ph.D., CEO,** Sri Krishna Institutions, Sri Krishna Arts and Science College, Coimbatore.

I convey my profound gratitude to **Dr. R. Jagajeevan MBA., Ph.D., Principal,** Sri Krishna Arts and Science College for giving me this opportunity to undergo this Capstone Project work.

It is my prime to solemnly express my sense of gratitude to **Dr. K. S. Jeen Marseline, M.C.A., M.Phil., Ph.D., Head and Dean,** Department of IT & Cognitive Systems, Sri Krishna Arts and Science College.

I would like to extend my thanks and unbound sense for the timely help and assistance given by **Mrs. VALARAMATHI M.C.A., M.Phil., Assistant Professor,** Department of IT & Cognitive Systems, Sri Krishna Arts and Science College in completing the Capstone Project work. Her remarkable guidance at every stage of my work was coupled with suggestion and motivation.

I take this opportunity to thank my parents and friends for their constant support and encouragement throughout this training.

HARI NITHIN S

PRAVEEN KUMAR M

CHAPTER 1

PROBLEM STATEMENT

Existing voice assistants such as Google Assistant, Alexa, and Siri provide convenience but remain limited in several ways: they often forget conversational context, offer restricted customization, lack strong accessibility support for visually impaired users, and pose data privacy concerns due to closed ecosystems. Open-source alternatives exist but usually fall short in terms of scalability, advanced AI capabilities, and professional-grade user interfaces.

This gap highlights the need for a scalable, intelligent, and secure AI Voice Assistant that goes beyond basic command recognition. The system should provide context-aware and sentiment-sensitive responses, support multi-platform integration, offer advanced features like web automation and document analysis, and ensure accessibility for all users while maintaining strong security and privacy controls.

INTRODUCTION

The **AI Voice Assistant** is a modern full-stack application that makes using technology easier, faster, and more accessible. It allows people to control and interact with computers through voice commands, providing quick responses and smooth integration across different platforms.

Unlike basic voice assistants that only understand fixed commands, this system uses **Google's Generative AI (Gemini)**. That means it can hold more natural, context-aware conversations, understand different types of input, and even learn and adapt over time. A key focus is making it especially useful for **visually impaired users**, ensuring that it's accessible to everyone.

The project is built with the latest technology stack:

- **Frontend:** Next.js and React for a clean and interactive interface
- **Backend:** FastAPI (Python) for powerful and efficient server operations
- **Database:** Firebase Firestore for real-time data handling

- **APIs & Tools:** Web Speech API, Gemini API, and secure authentication inspired by Stripe

Main Features

- Understands and processes voice commands with high accuracy
- Gives intelligent responses that adapt to user preferences and emotions
- Provides secure login, user profiles, and personalized settings
- Includes advanced tools like web scraping, automation, and data analytics

Vision

This project is designed to close the gap between **human communication and digital automation**. It's built to be **scalable, reliable, and ready for the future** — with potential upgrades like mobile apps, and enterprise-level features.

CHAPTER – 2

SYSTEM STUDY

2.1 REVIEW ON EXISTING WORK

Current voice assistants like Google Assistant, Alexa, and Siri provide convenience but are limited by platform lock-in, restricted customization, and data privacy concerns. Open-source alternatives often lack scalability, advanced AI models, and professional-grade interfaces.

However, most existing voice assistant systems face several challenges:

- **Poor conversational memory** — they often forget context after a single interaction.
- **Limited API integration** — making it hard to connect with external services and applications.
- **Weak accessibility support** — especially for users with special needs, such as the visually impaired.
- **Restricted automation and file handling** — they lack advanced capabilities like web automation or document processing.

These limitations highlight the growing need for a **customizable, AI-powered, and secure voice assistant** — one that not only offers advanced features but also keeps the interface simple, inclusive, and user-friendly.

2.2 Proposed Work

The proposed AI Voice Assistant is designed to overcome the existing limitations of traditional voice-based applications by integrating advanced features that make it more intelligent, secure, and user-friendly. The system aims to deliver an adaptive and productive experience through the following enhancements:

1. Integration of Gemini API for Contextual Responses and Learning
   The assistant leverages the Gemini API to provide context-aware, human-like responses. Unlike static assistants, it continuously learns from user interactions, enabling it to adapt over time. This ensures conversations remain relevant, personalized, and aligned with the user's needs.

2. Real-Time Voice Recognition with Confidence Detection
   A high-accuracy speech-to-text engine will be employed to recognize user commands instantly. In addition, confidence-level detection will help the assistant validate uncertain inputs, reducing errors and prompting clarifications when required. This creates a smoother and more reliable user experience.

3. Multi-Platform Command Execution
   The assistant will support commands across different domains, including:

   - System operations (e.g., opening apps, controlling volume, file navigation)
   - Web-based tasks (e.g., performing searches, retrieving information, scheduling events)
   - Automation workflows (e.g., triggering scripts, managing IoT devices, or handling repetitive processes).
     This multi-platform support ensures versatility and practical usage in day-to-day tasks.

4. Secure Firebase Authentication with Google OAuth
   To ensure strong user authentication and data privacy, the system will integrate Firebase Authentication alongside Google OAuth 2.0. This enables users to sign in securely using trusted credentials while safeguarding personal data.

5. Modern Glassmorphic UI with Accessibility Compliance
   The frontend will feature a glassmorphism-based design, ensuring a sleek and futuristic appearance. At the same time, accessibility guidelines (such as WCAG standards) will be followed to make the interface inclusive for users with visual or cognitive impairments, ensuring usability for a wider audience.

6. Enhanced Productivity Features: Web Scraping, PDF Analysis, and Browser Automation
   To extend beyond basic voice commands, the assistant will incorporate advanced productivity tools:

   - Web Scraping: Extracting structured data from websites for research or analysis.
   - PDF Analysis: Summarizing, extracting text, and answering queries from PDF documents.
   - Browser Automation: Executing tasks like auto-filling forms, managing tabs, and performing repetitive browsing actions.

These capabilities allow users to automate time-consuming processes, making the assistant valuable for students, professionals, and researchers alike.

2.3 Tools Used

The development of the AI Voice Assistant involves a combination of modern frameworks, libraries, and platforms to ensure scalability, efficiency, and a smooth user experience. The following tools and technologies are utilized:

1. **Frontend**
    - Next.js (15.2.4): Provides a powerful React-based framework with server-side rendering (SSR) and static site generation (SSG), ensuring fast performance and SEO optimization.
    - React 19: Core UI library for building modular, component-driven interfaces with improved concurrency features.
    - TypeScript 5+: Ensures type safety, scalability, and better maintainability in large codebases.
    - Tailwind CSS: Utility-first CSS framework for building responsive, consistent, and modern designs with minimal effort.
    - Radix UI: Accessible and customizable UI primitives to speed up the development of complex components.
    - Framer Motion: Enables smooth animations and transitions, making the UI more interactive and engaging.
2. **Backend**
    - FastAPI (0.104.1): High-performance Python web framework known for its speed, asynchronous capabilities, and easy API development.
    - Python 3.9+: Core backend programming language for implementing AI models, automation, and data processing.
    - Uvicorn (with WebSocket support): ASGI server for handling asynchronous requests and real-time communication (crucial for voice interaction and live responses).
3. **AI Engine**

- Google Generative AI (Gemini): Powers natural language understanding and contextual response generation for intelligent conversations.
- Web Speech API: Enables voice input and output functionalities, including speech-to-text and text-to-speech for real-time interaction.

4. **Automation**
   - Selenium: Used for web automation and testing tasks such as filling forms, scraping, and handling browser actions.
   - Playwright: Modern browser automation framework providing faster, reliable, and cross-browser automation for productivity features.
5. **Database**
   - Firebase Firestore: Cloud-based NoSQL database for real-time storage of user data, authentication records, and command history.
   - SQLite: Lightweight local database used for AI-specific features like caching context, storing embeddings, and offline functionality.
6. **Version Control & Deployment**
   - Git: Version control system for tracking changes and enabling collaborative development.
   - Vercel: Deployment platform optimized for Next.js, offering serverless hosting and fast global edge delivery.
   - Docker: Containerization platform ensuring environment consistency and easy deployment across systems.
   - Railway: Cloud platform for hosting backend services, APIs, and databases with seamless CI/CD integration.

CHAPTER – 3

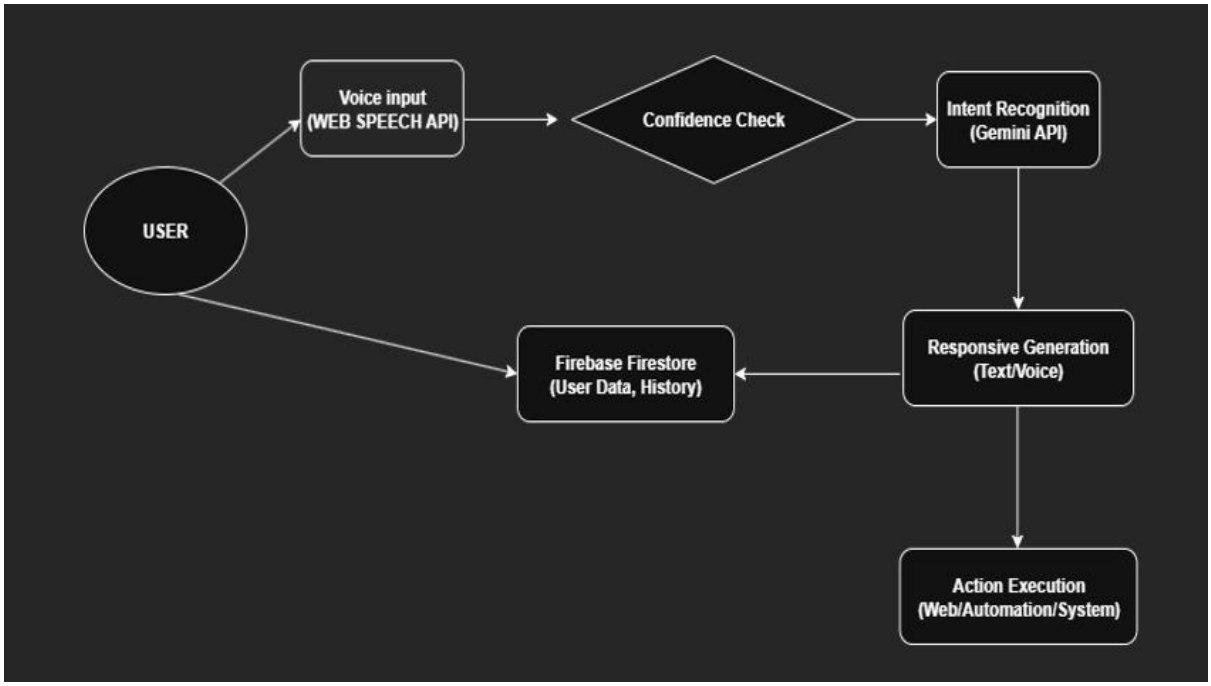
SYSTEM DESIGN

DATA FLOW DIAGRAM :

The Data Flow Diagram shows how the AI Voice Assistant works step by step. First, the **user speaks a command**, and the system captures it using the **Web Speech API**. The input then goes through a **confidence check** to make sure the speech is understood correctly. Next, the **Gemini API** figures out the user's intent — basically, what the user really means.

After that, the system **creates a response** (either as text or voice) and gives it back to the user. If the command involves an action, like opening a website or running an automation, the **Action Execution module** carries it out. At the same time, the system saves the user's data and history in **Firebase Firestore** so it can learn and improve over time.

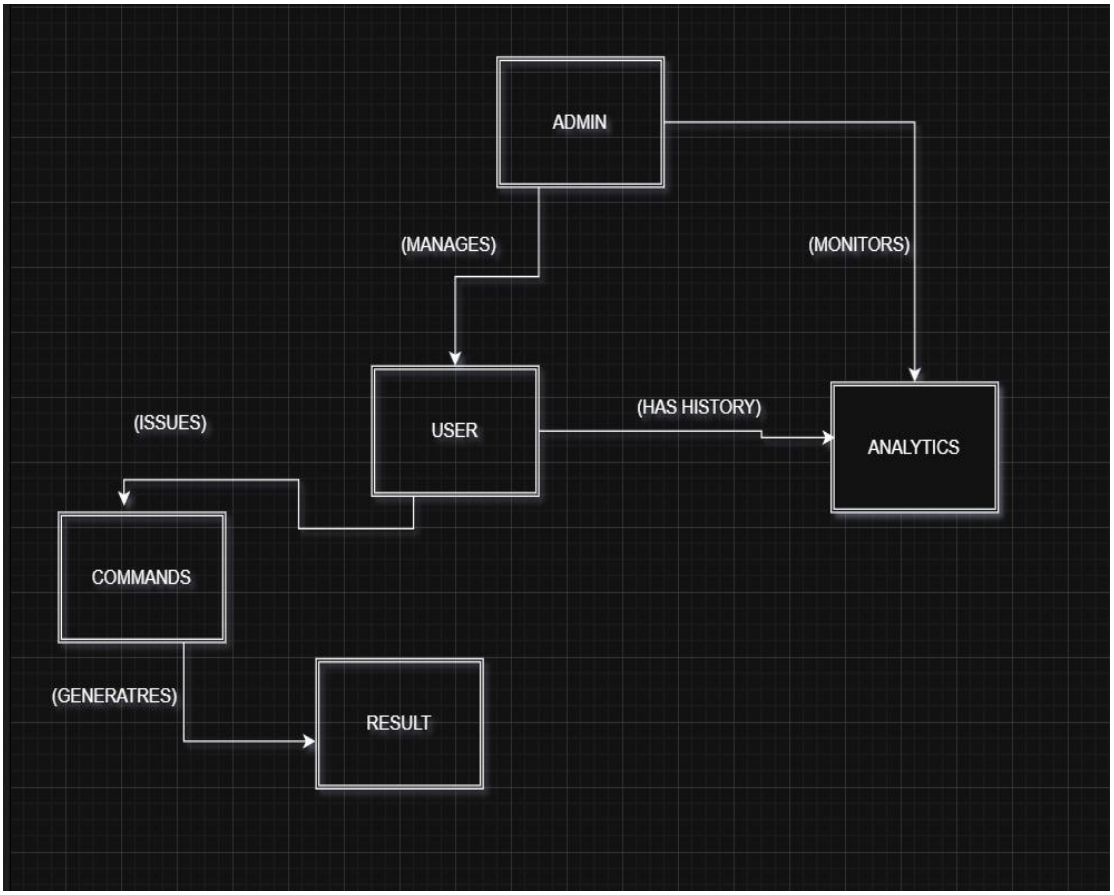
In short, the diagram explains how data flows from the user's voice, gets processed by AI, and finally turns into useful responses or actions, while also keeping track of history.



ER DIAGRAM

The Entity-Relationship (ER) diagram provides a detailed view of the database structure of the AI Voice Assistant system. The primary entities include Admin, User, Command, Response, and Analytics. A User can issue multiple Commands, and each Command generates a Response. The User also has a history that is stored and analyzed in Analytics. The Admin is responsible for managing Users and monitoring Analytics to ensure smooth operation of the system.

Relationships such as “user issues command,” “command generates response,” “user has history,” and “admin monitors analytics” define how the entities interact with one another. This ER diagram ensures that the system design supports the required features, such as user management, command processing, response generation, and performance monitoring, while maintaining data accuracy and accessibility.



CHAPTER – 4

MODULES

Module 1: Voice Processing & Recognition

Overview

This module handles real-time speech input, converting user voice into accurate text commands. It ensures smooth interaction by verifying confidence levels and providing a visual representation of speech activity.

Key Features

- Real-time voice capture and transcription.
- Confidence-level detection with user confirmation.
- Canvas-based waveform visualization for better interaction feedback.

Technical Implementation

- **Frontend Components:** VoiceRecorder (voice input), WaveformVisualizer (canvas-based UI).
- **APIs/Services:** Web Speech API for speech-to-text processing.

Module 2: AI Command Processing

Overview

This module is the intelligence core of the system. It identifies user intent, maintains conversational memory, and generates context-aware, sentiment-sensitive responses.

Key Features

- Intent recognition using Gemini API.
- Contextual understanding with memory-based conversation.
- Sentiment-aware and adaptive responses.

Technical Implementation

- **Backend Services:** Gemini API integration for NLP.
- **Logic Layer:** Context handler, sentiment analyzer.

Module 3: Authentication & User Management

Overview

This module ensures secure login and personalized user profiles. It manages account data, session security, and user-specific settings.

Key Features

- Firebase authentication via Email/Google OAuth.

- Secure session handling with JWT.
- User profile management (account details, settings, logout).

Technical Implementation

- **Frontend Components:** LoginForm, ProfilePage, SettingsManager.
- **Security:** Firebase Auth, JWT-based token handling.

Module 4: Web Integration & Automation

Overview

This module expands the assistant's capability by enabling web interactions and automated workflows. It can scrape data, analyze documents, and perform browser-based automation.

Key Features

- Smart web scraping using Selenium/Playwright.
- PDF and document analysis.
- Browser automation for repetitive workflows.

Technical Implementation

- **Backend Services:** Selenium/Playwright for scraping and automation.
- **Utility Layer:** Document parser, workflow executor.

Module 5: Analytics & History

Overview

This module provides insights into system usage and user activity. It tracks command history, generates statistics, and ensures user privacy with data management options.

Key Features

- Command history tracking and retrieval.

- Usage statistics with charts and insights.
- Privacy controls for deleting or exporting history.

Technical Implementation

- **Frontend Components:** HistoryViewer, AnalyticsDashboard.
- **Backend Services:** Firestore database for storing history and analytics.

Module 6: Admin Dashboard

Overview

The Admin Dashboard acts as the control panel for administrators. It allows them to monitor system activity, manage users, and configure system-wide settings securely.

Key Features

- Monitor system activity in real time.
- Manage users and API keys.
- Configure system-wide preferences and security policies.

Technical Implementation

- **Frontend Components:** AdminPanel, UserManagement, SystemConfig.
- **Backend Services:** Admin APIs for monitoring, user management, and configuration.

CHAPTER – 5

SYSTEM TESTING

4.1 SYSTEM TESTING

System testing was carried out in multiple phases to validate functionality and reliability. **Unit testing** focused on individual modules such as voice capture, authentication, and AI response generation, ensuring that each worked correctly in isolation. **Integration testing** validated the interaction between the frontend, backend, and Gemini API, confirming that data flowed smoothly across components. **User acceptance testing** was performed with sample users who executed real-world tasks such as logging in, issuing commands, and retrieving responses to verify that the system met expected requirements.

4.2 SECURITY TESTING

Security testing was conducted to confirm the system's protection mechanisms. Firebase Authentication and OAuth token handling were evaluated for safe user login and session management. Communication between modules was tested for encryption to ensure secure data transfer. API keys and Firebase security rules were reviewed to prevent unauthorized access. The results confirmed strong safeguards against data breaches and unauthorized activities.

4.3 PERFORMANCE TESTING

Performance testing measured the system's responsiveness and stability under load. Stress tests simulated multiple concurrent users issuing voice commands to evaluate latency across the voice-to-response pipeline. Load testing assessed how well the system managed real-time interactions, ensuring minimal delays during peak traffic. The results demonstrated that the system is capable of handling concurrent usage while maintaining real-time performance and reliability.

CHAPTER – 6

CONCLUSION AND FUTURE ENHANCEMENTS

The development of the AI Voice Assistant showcases how modern AI technologies and advanced web frameworks can come together to create an intelligent, scalable, and accessible digital assistant. The system meets its primary objectives by combining voice recognition, AI-powered responses, user authentication, web automation, and analytics into a unified platform. Users benefit from seamless voice-based interaction, accurate responses, and personalized features, while administrators gain control through monitoring tools and system configuration options. Built with React, FastAPI, Firebase, and Gemini API, the assistant ensures scalability, real-time performance, and secure operations, making it a robust solution for both personal and professional use.

Looking ahead, the AI Voice Assistant has strong potential for continued growth through planned enhancements such as a dedicated **mobile companion app** built with React Native, **offline/local**

processing for faster responses, and **personalized voice cloning** for more natural text-to-speech interactions. Expanding **multi-language support** will make the system globally accessible, while **team collaboration features** like multi-user sessions will extend its utility in professional and academic environments. Additionally, the integration of **advanced AI models beyond Gemini** can enhance intelligence and adaptability, keeping the system at the forefront of innovation. With these improvements, the AI Voice Assistant is set to evolve into a next-generation platform for personal, academic, and enterprise applications.

CHAPTER – 7

APPENDIX

i) Sample Code

Firebase core (simplified):

ts

```
// lib/firebase.ts
import { initializeApp, getApps, getApp } from "firebase/app"
import { getAuth } from "firebase/auth"
import { getFirestore } from "firebase/firestore"
```

```
const firebaseConfig = {
  apiKey: process.env.NEXT_PUBLIC_FIREBASE_API_KEY || "AIzaSyC4...",
  authDomain: process.env.NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN || "voice-assistant-f6fcc.firebaseapp.com",
  projectId: process.env.NEXT_PUBLIC_FIREBASE_PROJECT_ID || "voice-assistant-f6fcc",
  // ...
}

let app: any; let auth: any; let db: any;

export function getFirebaseApp() { app = app ?? (getApps().length ? getApp() : initializeApp(firebaseConfig)); return app }

export function getFirebaseAuth() { auth = auth ?? getAuth(getFirebaseApp()); return auth }

export function getFirebaseFirestore() { db = db ?? getFirestore(getFirebaseApp()); return db }
```

Authentication Flow (simplified):

tsx

```
// components/auth/auth-context.tsx
"use client"
import { createContext, useContext, useEffect, useState } from "react"
import { User, onAuthStateChanged } from "firebase/auth"
import { getFirebaseAuth } from "@/lib/firebase"
import { userHistoryService } from "@/lib/user-history"

export function AuthProvider({ children }: { children: React.ReactNode }) {
```

```
  const [user, setUser] = useState<User | null>(null)
  const [sessionId, setSessionId] = useState<string | null>(null)
  const auth = getFirebaseAuth()

  useEffect(() => {
    const unsubscribe = onAuthStateChanged(auth, async (user) => {
      setUser(user)
      if (user && !sessionId) {
        const newSessionId = await userHistoryService.startSession(user.uid)
        setSessionId(newSessionId)
      } else if (!user && sessionId) {
        await userHistoryService.endSession(sessionId)
        setSessionId(null)
      }
    })
    return () => unsubscribe()
  }, [sessionId])

  return <AuthContext.Provider value={{ user, loading: !user && !sessionId, sessionId, signOut: async () => { if (sessionId) await userHistoryService.endSession(sessionId); await auth.signOut() } }}>{children}</AuthContext.Provider>
}
```

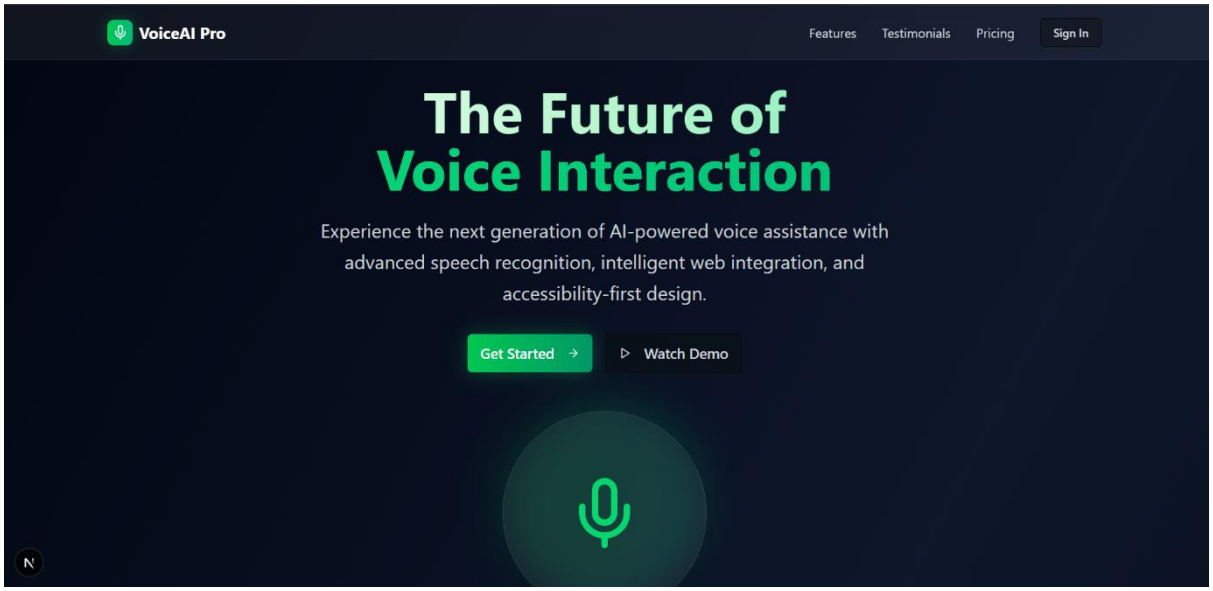
tsx

```
// components/auth/login-form.tsx
import { signInWithEmailAndPassword, signInWithPopup, createUserWithEmailAndPassword, updateProfile, GoogleAuthProvider } from "firebase/auth"
```

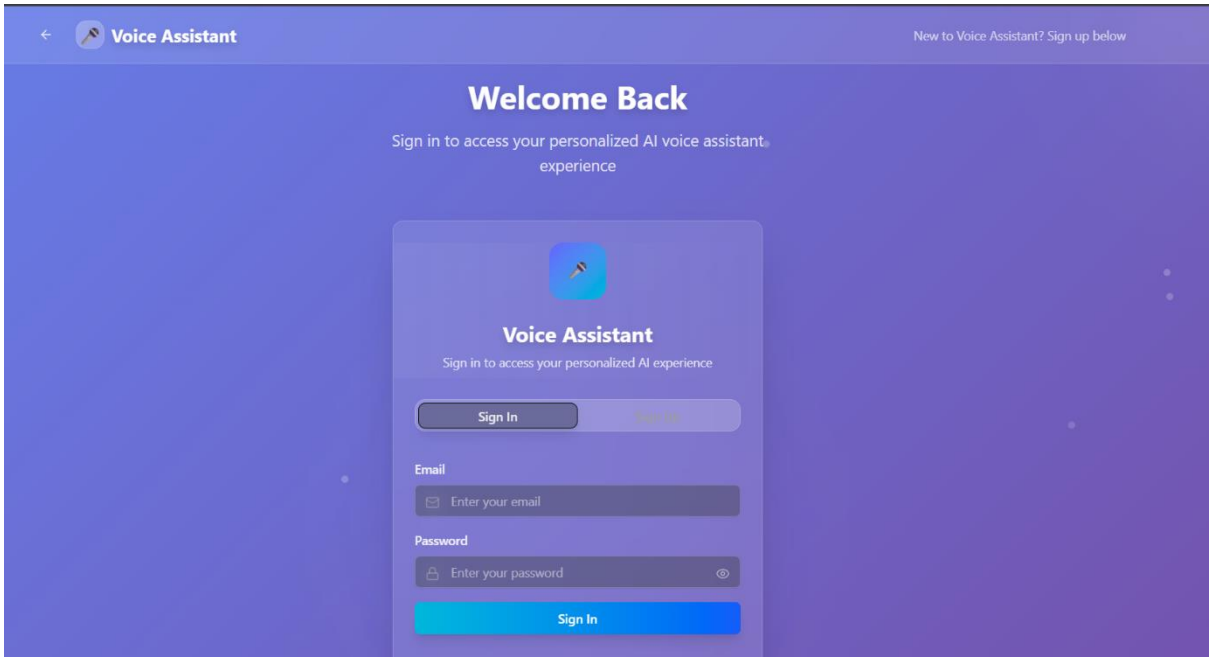
```
import { getFirebaseAuth } from "@/lib/firebase"


export function LoginForm({ onSuccess }: { onSuccess?: () => void }) {
  const auth = getFirebaseAuth()
  const googleProvider = new GoogleAuthProvider()


  const handleEmailAuth = async (isSignUp: boolean) => {
    if (isSignUp) {
      const cred = await createUserWithEmailAndPassword(auth, email, password)
      if (displayName) await updateProfile(cred.user, { displayName })
    } else {
      await signInWithEmailAndPassword(auth, email, password)
    }
    onSuccess?.()
  }


  const handleGoogleAuth = async () => {
    await signInWithPopup(auth, googleProvider)
    onSuccess?.()
  }
}
```

ii) Screenshots (Sample Layouts)

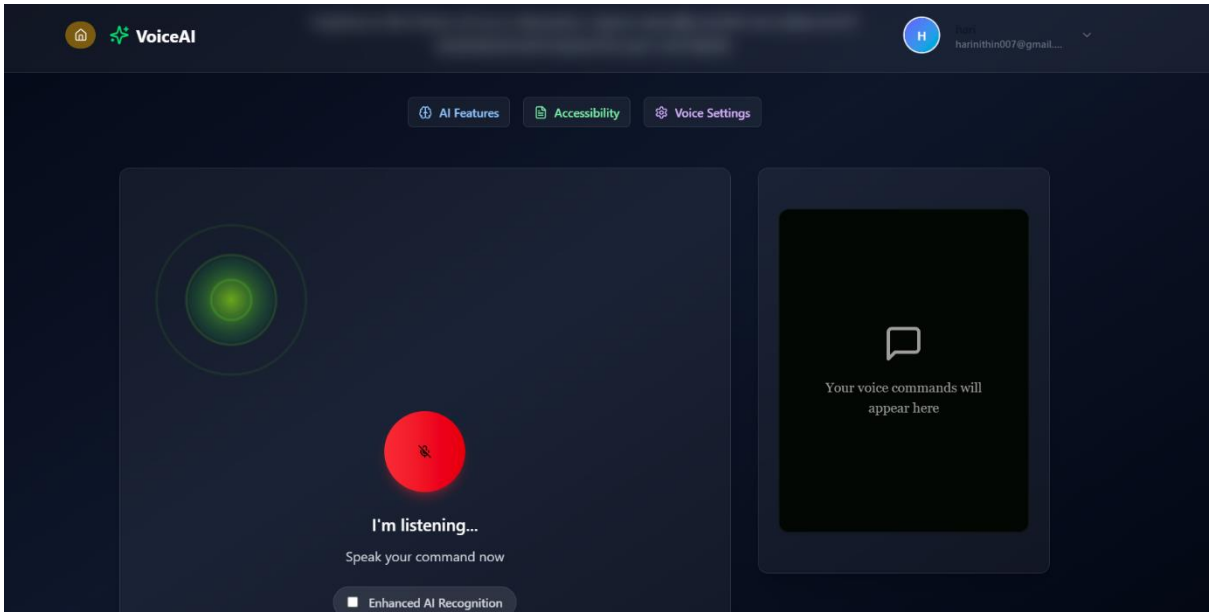
Landing Page → Gradient background with CTA.



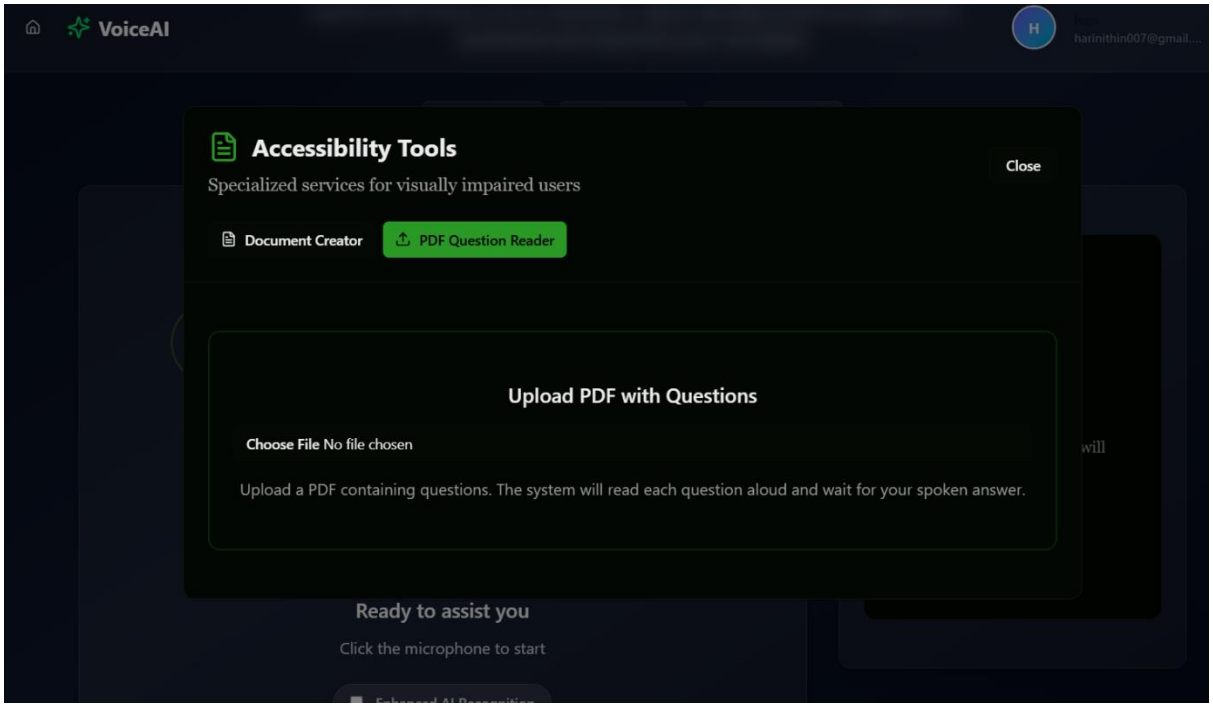
Login Page → Glassmorphic login card with OAuth.



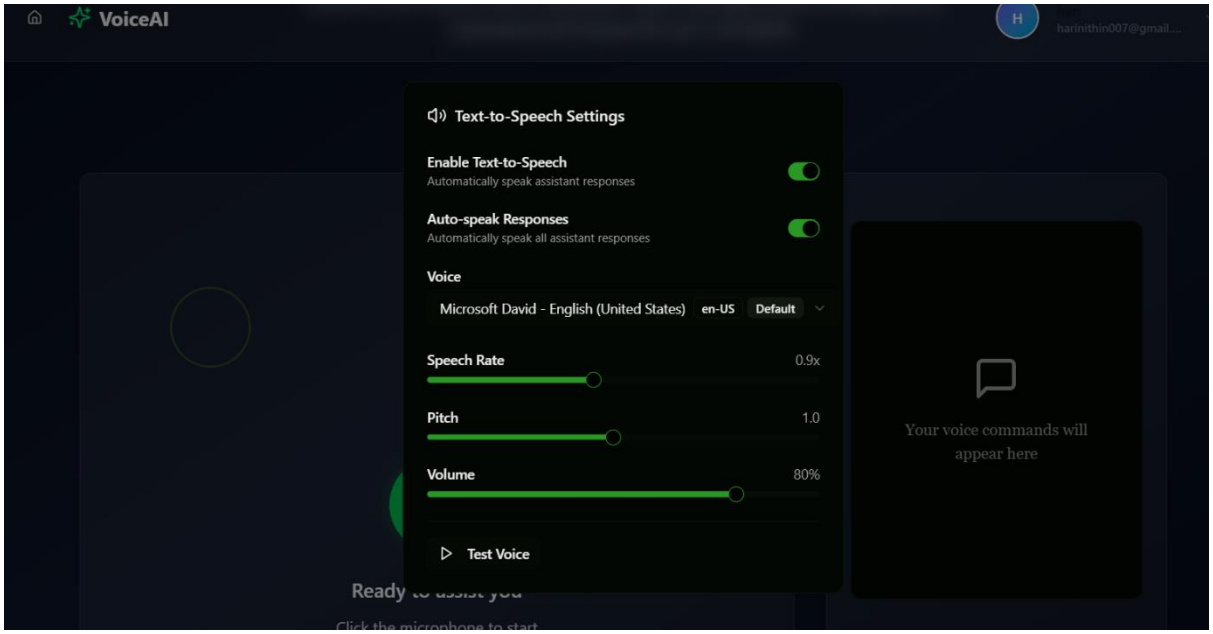
Assistant Interface → Voice waveform, AI response panel.



Accessibility Dashboard → User history and statistics.



Settings Page → Voice & privacy controls.


VoiceAI
H
harinithin007@gmail....
Text-to-Speech Settings
Enable Text-to-Speech
Automatically speak assistant responses
Auto-speak Responses
Automatically speak all assistant responses
Voice
Microsoft David - English (United States)
en-US
Default
Speech Rate
0.9x
Pitch
1.0
Volume
80%
Test Voice
Your voice commands will appear here
Ready to assist you
Click the microphone to start